



Московский государственный университет имени
М.В.Ломоносова
Факультет вычислительной математики и кибернетики
Кафедра автоматизации систем вычислительных комплексов

Рогинко Алла Дмитриевна

**Исследование и реализация жадных
алгоритмов
для задачи распределения таблиц
классификации
по стадиям конвейера СПУ**

Курсовая работа

Научный руководитель:
Доцент, к.ф.-м.н.
Капитонова Алла Петровна
Научный консультант:
Никифоров Никита Игоревич

Москва, 2026

Аннотация

Исследование и реализация жадных алгоритмов
для задачи распределения таблиц классификации
по стадиям конвейера СПУ

Рогинко Алла Дмитриевна

Рассматривается задача размещения логических match-action таблиц P4 по стадиям конвейера архитектуры RMT при учёте графа зависимостей таблиц и ограничений аппаратных ресурсов. Проводится обзор жадных эвристик (FFD, FFL, FFLS, MCF), описывается декомпозиция задачи на этапы топологической упорядоченности, проверки ресурсов и поиска размещения. Представлены программная реализация многопроходного варианта FFD с поддержкой горизонтального разрезания таблиц и эталонной реализации FFLS, а также методика экспериментального сравнения по числу занятых стадий и времени работы на синтетических конфигурациях. Сформулированы выводы о применимости рассмотренных подходов при различных размерах и плотности зависимостей графа.

Abstract

Содержание

1	Введение	5
2	Предметная область	6
2.1	Сетевые процессорные устройства	6
2.2	Архитектура RMT	6
2.3	Программа обработки пакетов и язык P4	7
2.4	Граф зависимостей и виды зависимостей внутри таблиц P4	7
3	Цель работы и постановка задачи	9
3.1	Актуальность работы	9
3.2	Цель работы	9
3.3	Постановка задачи	9
4	Обзор существующих алгоритмов	11
4.1	NP-трудность задачи размещения таблиц	11
4.2	Точные методы: целочисленное линейное программирование (ILP)	11
4.3	Жадные эвристики	11
4.4	Выводы по обзору	12
5	Реализованные алгоритмы	14
5.1	Горизонтальное разрезание таблиц	14
5.2	Многопроходный FFD	14
5.3	FFLS	15
6	Описание архитектуры реализованных алгоритмов	16
6.1	Структура кода и основные модули	16
7	Экспериментальное исследование	18
7.1	Методика экспериментального исследования	18
7.2	Анализ полученных результатов	18
7.2.1	Результаты сравнения алгоритмов по времени выполнения	18
7.2.2	Результаты сравнения алгоритмов по количеству стадий . .	18

8 Заключение	20
Список литературы	21

1 Введение

В современных сетях возникает необходимость в гибкой обработке пакетов. Необходимо быстро внедрять новые протоколы и сервисы без замены оборудования. Решением этого вопроса является использование программируемых сетевых устройств с архитектурой RMT с языком описания пакетной обработки P4. Такие устройства реализуют конвейерную обработку, состоящую из последовательных match-action стадий.

При компиляции P4-программы для архитектуры RMT нужно распределить логические таблицы по стадиям конвейера. От этого решения зависят число задействованных стадий и, следовательно, задержка обработки пакета. Для успешного размещения следует учитывать ограничения архитектуры аппаратуры и порядок применения таблиц, заданный графом зависимостей (Table Dependency Graph). На практике стремятся минимизировать число занятых стадий, поскольку оно непосредственно определяет задержку обработки пакета.

В данной работе рассматриваются жадные алгоритмы решения задачи размещения, анализируется их эффективность и быстрота выполнения, и предлагается модификация с поддержкой горизонтального разрезания таблиц.

2 Предметная область

2.1 Сетевые процессорные устройства

Сетевые процессорные устройства (СПУ) — это специализированные системы, предназначенные для обработки сетевого трафика на высоких скоростях. Проще говоря, такие устройства должны очень быстро принимать пакеты, анализировать их и принимать решение о дальнейшей обработке. В отличие от традиционных сетевых устройств, где логика работы часто жёстко зафиксирована на уровне аппаратуры, программируемые СПУ позволяют изменять алгоритмы обработки пакетов и адаптировать их под разные задачи. Отличительной чертой современных СПУ является наличие программируемой логики обработки пакетов, что позволяет адаптировать устройство под новые протоколы и сетевые функции без замены аппаратного обеспечения.

2.2 Архитектура RMT

Одной из архитектур эталонной для построения высокопроизводительных программируемых сетевых устройств является RMT (Reconfigurable Match Tables). Основная идея этой архитектуры заключается в использовании набора настраиваемых таблиц сопоставления (match tables), которые применяются на разных стадиях конвейера обработки пакетов. На каждой стадии на основе выбранного ключа выполняется поиск в одной или нескольких таблицах, где правила сопоставления определяют, какие действия должны быть выполнены над пакетом. Эти действия могут включать изменение заголовков, выбор следующей стадии обработки или передачу пакета на выходной интерфейс. Благодаря такой структуре архитектура остаётся гибкой и позволяет реализовывать различные алгоритмы обработки сетевого трафика.

Match-action стадии — последовательность одинаковых стадий. Каждая стадия содержит:

1. Таблицы точного поиска, реализованные на SRAM.
2. Таблицы тернарного и префиксного поиска, реализованные на TCAM.

3. Кроссбары для подачи произвольных полей из PHV на вход таблиц.
4. Action-блоки для параллельного изменения полей PHV.

2.3 Программа обработки пакетов и язык P4

Для описания поведения программируемого коммутатора используется язык P4 (Programming Protocol-independent Packet Processors). Программа на P4 определяет набор логических match-action таблиц, каждая из которых включает набор полей заголовка, по которым производится поиск и список возможных действий для них. P4 является аппаратно-независимым языком: одна и та же программа может компилироваться для различных устройств.

2.4 Граф зависимостей и виды зависимостей внутри таблиц P4

При компиляции P4-программы в конвейер RMT строится граф зависимостей таблиц (Table Dependency Graph, TDG). Вершинами графа являются логические таблицы, а рёбрами — зависимости между таблицами. Корректное размещение таблиц по стадиям конвейера должно удовлетворять всем видам зависимостей, иначе обработка пакетов будет нарушена. Выделяют четыре основных типа зависимостей:

Match-зависимость возникает, когда таблица A изменяет поле, а таблица B выполняет поиск по этому полю. Эта зависимость соответствует классической зависимости «чтение после записи». Для корректной работы необходимо, чтобы A полностью завершила выполнение своих действий до того, как B начнёт поиск. Поэтому, таблицы с match-зависимостью не могут располагаться в одной стадии конвейера.

Action-зависимость появляется, когда две таблицы A и B изменяют одно и то же поле, и важен результат, полученный в таблице B . При размещении допускается нахождение A и B в одной стадии.

Successor-зависимость заключается в следующем: выполнение таблицы B зависит от результата таблицы A . Таблицы с successor-зависимостью могут размещаться в одной стадии.

Обратная match-зависимость возникает, когда таблица A выполняет поиск по полю, которое впоследствии изменяется таблицей B . Также требует строгого разделения стадий: A должна быть размещена в стадии, строго меньшей, чем B .

3 Цель работы и постановка задачи

3.1 Актуальность работы

Рост скорости сетевого трафика и требование быстро менять поведение коммутаторов без замены оборудования делают актуальной программируемую сетевую обработку. Компиляция Р4-программы в конвейер RMT включает этап размещения логических таблиц по стадиям при ограничениях памяти и графе зависимостей. Качество этого этапа влияет на число занятых стадий и задержку обработки пакета, поэтому исследование эвристик размещения и их реализации для типовых ограничений TDG сохраняет практический интерес для разработчиков компиляторов и сетевого оборудования.

3.2 Цель работы

Целью данной работы является исследование жадных алгоритмов решения задачи размещения логических таблиц классификации по стадиям конвейера RMT-коммутатора, а также разработка программной реализации этих алгоритмов для оценки ресурсных затрат Р4-программ.

3.3 Постановка задачи

Формально задача размещения таблиц классификации может быть сформулирована следующим образом.

Дано:

1. Множество логических таблиц $T = \{t_1, t_2, \dots, t_n\}$, каждая из которых характеризуется:
 - типом сопоставления $\text{match_type}(t_i) \in \{\text{exact}, \text{ternary}, \text{lpm}\}$;
 - шириной ключа $w(t_i)$ (в битах);
 - максимальным количеством записей $e(t_i)$;
 - списком зависимостей от других таблиц с указанием типа зависимости $\text{dep_type} \in \{\text{match}, \text{action}, \text{successor}, \text{reverse_match}\}$.

2. Описание аппаратной архитектуры RMT:

- количество match-action стадий S_{total} ;
- количество блоков SRAM на стадию B_{sram} , ширина блока W_{sram} (бит), глубина блока D_{sram} (количество записей);
- количество блоков TCAM на стадию B_{tcam} , ширина блока W_{tcam} (бит), глубина блока D_{tcam} (количество записей).

Требуется: Построить схему размещения, которая каждой таблице (или каждому фрагменту таблицы при горизонтальном разрезании) ставит в соответствие номер стадии конвейера, такое, что:

1. Выполнены все аппаратные ограничения.
2. Соблюдены все типы зависимостей между таблицами.
3. Количество использованных стадий минимально.

4 Обзор существующих алгоритмов

4.1 NP-трудность задачи размещения таблиц

Задача размещения логических таблиц классификации по стадиям конвейера RMT является вычислительно сложной. В работе [?] доказано, что эта задача является NP-трудной. Более того, авторы показывают, что для ряда моделей не существует алгоритма, который решал бы её точно за полиномиальное время.

Следовательно, для практического применения при значительных объёмах входных данных (десятки и сотни таблиц) необходимо использовать приближённые методы, в частности, жадные эвристики.

4.2 Точные методы: целочисленное линейное программирование (ILP)

Точное решение задачи размещения может быть получено с помощью целочисленного линейного программирования (Integer Linear Programming, ILP). В работе [?] предложена ILP-модель, включающая:

- переменные, указывающие, в каких стадиях размещены таблицы;
- ограничения, гарантирующие соблюдение ёмкости памяти и зависимостей;
- целевую функцию, минимизирующую количество использованных стадий.

ILP позволяет найти оптимальное размещение, однако время его работы экспоненциально зависит от числа таблиц. Эксперименты из [?] показывают, что для программы с 24 логическими таблицами ILP-решатель затрачивает около 6300 секунд на поиск оптимума. Для более крупных программ время становится неприемлемым для практического использования, особенно при необходимости перекомпиляции в реальном времени.

4.3 Жадные эвристики

В связи с NP-трудностью задачи для практических применений разработаны жадные эвристики, которые работают за полиномиальное время и обеспе-

чивают приемлемое качество размещения. В литературе [?] выделяют четыре основных типа эвристик:

FFD (First Fit Decreasing) – классический алгоритм упаковки. Таблицы сортируются по убыванию количества требуемых блоков памяти. Затем каждая таблица размещается в первую стадию, в которой для неё достаточно свободной памяти. FFD не учитывает топологический порядок зависимостей, что может приводить к невозможности разместить корректно все таблицы при наличии match-зависимостей.

FFL (First Fit by Level) – перед сортировкой вычисляются уровни таблиц: уровень таблицы равен длине самого длинного пути в графе зависимостей от любого источника. Таблицы сортируются по возрастанию уровня, а внутри уровня – в произвольном порядке. Это гарантирует, что предки обрабатываются раньше потомков, однако из-за произвольного порядка внутри уровня это может приводить к неэффективному использованию памяти на одной стадии.

FFLS (First Fit by Level and Size) – модификация FFL, в которой внутри одного уровня таблицы дополнительно сортируются по убыванию размера. Данный подход сочетает преимущества топологического порядка и плотной упаковки.

MCF (Most Constrained First) – таблицы сортируются по степени «ограниченности»: в первую очередь обрабатываются таблицы, которые могут быть размещены только в определённом типе памяти (например, тернарные – только в ТСАМ) или имеют наибольшую ширину ключа.

4.4 Выводы по обзору

Таким образом, задача размещения таблиц является NP-трудной. Точные методы (ILP) обеспечивают оптимальность, но неприменимы для больших программ из-за экспоненциального времени работы. Жадные эвристики работают быстрее. Однако классические жадные эвристики имеют недостаток – они могут

не найти размещение для всех таблиц из-за однопроходности. Поэтому актуальной является разработка многопроходных и гибридных подходов. В последующих разделах будет представлена реализация многопроходного FFD и FFLS, а также результаты их экспериментального сравнения по времени работы и количеству использованных стадий.

5 Реализованные алгоритмы

В настоящем исследовании предлагается модификация – многопроходный FFD с поддержкой горизонтального разрезания таблиц, которая устраняет этот недостаток и позволяет достичь **полной успешности размещения** для всех протестированных конфигураций.

5.1 Горизонтальное разрезание таблиц

Если таблица не помещается целиком ни на одной допустимой стадии, программа размещения может разбить её на несколько фрагментов с последующим согласованным выполнением (horizontal table split). Это увеличивает число узлов TDG и усложняет проверку зависимостей, но позволяет избежать тупика при жёстких лимитах SRAM/TCAM. В реализации модуль горизонтального разрезания вызывается планировщиком, когда ни одна допустимая по TDG стадия не содержит достаточной остаточной квоты памяти при выполнении предиката допустимости. Сплиттер порождает фрагменты с обновлёнными требованиями ресурсов и дополнительными ограничениями предшествования так, чтобы части одной логической таблицы выполнялись последовательно с согласованием ключей; политика дробления задаётся конфигурацией и не затрагивает интерфейс остальных модулей (см. раздел 6).

— при горизонтальном разрезании таблицы все её фрагменты должны располагаться в стадиях с номерами, образующими строго возрастающую последовательность без пропусков: $s_1 < s_2 < \dots < s_k$.

— если таблица t_i (предок) разрезана на фрагменты, а таблица t_j (потомок) связана с t_i match-зависимостью, то все фрагменты t_i должны быть размещены в стадиях, строго меньших, чем все фрагменты t_j : $\max \phi_{\text{frag}}(t_i) < \min \phi_{\text{frag}}(t_j)$.

5.2 Многопроходный FFD

Многопроходная реализация FFD строится как управляющий цикл над одношаговым размещением таблицы. Перед каждым проходом таблицы упорядочиваются по убыванию выбранной меры ресурсоёмкости; для текущей таблицы

последовательно перебираются номера стадий, начиная с минимально допустимого согласно TDG. Для пары (*таблица, стадия*) проверяются: размещение всех предшественников и типы зависимостей (включая ограничения «не в одной стадии» для match и обратной match-связи), совместимость с уже размещёнными на стадии таблицами, суммарная занятость SRAM/TCAM. При успехе таблица фиксируется; при неудаче после полного перебора стадий либо увеличивается число стадий-приёмников ресурсов, либо выполняется разрезание (см. п. 5.1). После завершения прохода выполняется анализ таблиц, оставшихся без размещения: при наличии таковых возможен следующий проход с обновлённым графом (после сплитов) или изменение параметров упорядочивания вплоть до исчерпания лимита итераций. Таким образом, многопроходность охватывает повторную подачу множества таблиц после частичных решений или дробления до стабилизации результата.

5.3 FFLS

Алгоритм FFLS (First Fit by Level and Size) в реализации сочетает топологический порядок с жадным first fit по стадиям. На входе вычисляются уровни вершин TDG (или эквивалентная топологическая стратификация); таблицы внутри каждого уровня упорядочиваются по убыванию ресурсоёмкости, после чего к каждой применяется тот же предикат допустимости и правило первой подходящей стадии, что и в FFD. Такой порядок согласует обход графа с направлением зависимостей и задаёт понятный эталон для сравнения с FFD без явного включения топологических уровней в правило упорядочивания последнего.

6 Описание архитектуры реализованных алгоритмов

6.1 Структура кода и основные модули

Для прототипирования алгоритмов размещения выбран язык Python 3: удобны графовые структуры, словари описаний таблиц и воспроизводимость экспериментов; перенос критичных по времени участков на компилируемый язык возможен без изменения логики размещения.

Для представления графа зависимостей и вычисления уровней используются структуры на основе стандартной библиотеки (`dict` списков смежности, очередь при обходе в ширину). Зависимость от сторонних пакетов минимизирована; при необходимости визуализации TDG допускается подключение `networkx`.

Компоненты комплекса. Логическая структура следующая:

1. модуль входных данных — загрузка описания набора таблиц, ключей и требований к SRAM/TCAM, списков рёбер TDG по четырём категориям зависимостей;
2. модуль валидации — проверка согласованности графа (отсутствие конвейерно неприемлемых циклов и пр.);
3. планировщик — реализации FFD с многопроходным режимом и FFLS (и опционально FFL/MCF для сравнения) через общий интерфейс *граф ресурсов* $\rightarrow$ *расписание по стадиям*;
4. модуль горизонтального разрезания (сплиттер) — см. раздел 5.1;
5. модуль отчёта — число стадий, профиль загрузки SRAM/TCAM и замеры времени.

Уровни абстракции. На каноническом уровне TDG задаётся в терминах логических таблиц P4 и классифицированных зависимостей; на планировочном — таблицы снабжаются числовой ресурсоёмкостью и упорядочиваются

выбранной эвристикой; на размещающем ведётся состояние конвейера: остаточные квоты и множества размещённых таблиц на стадии, операции допустимости, коммита и при необходимости отката.

Разделение ответственности. Модуль графовых операций инкапсулирует уровни (длину пути для FFL/FFLS), списки предшественников и классификацию ограничений «нельзя в одной стадии». Модуль ресурсов суммирует SRAM/TCAM и проверяет лимиты с учётом типа таблицы. Ядро FFD включает сортировку по убыванию веса; ядро FFLS вызывает сортировку внутри слоёв топологической декомпозиции. Многоходовой FFD со сплитами реализован как внешний цикл над одношаговым разместителем (согласование с разделом 5).

Интерфейсы эвристик. Эвристики реализуют общий метод `place(tdg, limits)` с отображением «таблица $\rightarrow$ номер стадии» и диагностикой (число итераций, число сплитов); добавление MCF или локального улучшающего прохода сводится к тому же интерфейсу. Лимиты стадий и квот памяти вынесены в отдельный объект для эмуляции разных целевых профилей RMT.

Сложность и время. Пусть n — число логических таблиц после возможных сплитов, m — верхняя граница числа стадий, d — среднее число зависимостей на таблицу. Проверка допустимости для пары (таблица, стадия) в наивной реализации — $O(d)$ к предшественникам и $O(k)$ конфликтов на стадии ($k \leq n$); перебор стадий даёт $O(m)$, размещение одной таблицы в худшем случае $O(m(d+k))$, для всего набора порядка $O(nm(d+n))$, что при $n, m \ll 10^3$ допускает интерактивные эксперименты. Память: $O(n+d)$ на граф, $O(m)$ на размещение и остатки ресурсов, $O(n)$ на вспомогательные структуры уровней.

На синтетических конфигурациях с сотнями таблиц время одного запуска, как правило, остаётся долями секунды; узким местом становятся генерация очень плотных TDG или полное трассирование решений.

7 Экспериментальное исследование

7.1 Методика экспериментального исследования

Цель экспериментов — сравнить жадные эвристики FFD с многопроходным режимом и горизонтальным разрезанием и FFLS по метрикам: (1) число задействованных стадий конвейера после успешного размещения; (2) максимальная и средняя утилизация SRAM и TCAM по стадиям; (3) процессорное время работы размещителя на одном экземпляре TDG. Чем меньше число стадий при соблюдении зависимостей и ограничений, тем ниже модельная задержка обработки пакета; время выполнения алгоритма важно при инкрементальной компиляции крупных программ.

Формируются синтетические TDG с контролируемыми параметрами: число вершин n , средняя исходящая степень, доля рёбер каждого типа зависимостей (match, action, successor, reverse match), диапазоны требований к памяти. Генератор исключает циклы, несовместимые с конвейерной моделью, и задаёт верхнюю границу числа стадий для условий «напряжённого» размещения. Для каждой тройки (n , плотность, соотношение типов) выполняется серия из k экземпляров (например, $k = 30$) с фиксированными зёрнами ГПСЧ. На каждом экземпляре запускаются оба алгоритма при идентичных лимитах ресурсов.

7.2 Анализ полученных результатов

7.2.1 Результаты сравнения алгоритмов по времени выполнения

Отношение времени FFLS к времени FFD на общих входах зависит от затрат на вычисление уровней и сортировок внутри слоёв: при малых n различия малы; при n порядка сотен дополнительные расходы FFLS остаются сортировочно-линейными и обычно не выходят за доли секунды в интерпретируемой реализации.

7.2.2 Результаты сравнения алгоритмов по количеству стадий

При низкой плотности TDG и умеренных квотах FFD и FFLS, как правило, дают близкое число стадий. С ростом плотности match и обратных match-

ограничений FFLS часто даёт меньше конфликтных попыток благодаря согласованию обхода с направлением зависимостей в TDG; при дефиците TCAM на ранних уровнях преимуществом может обладать FFD из-за раннего размещения крупных таблиц.

Многопроходный режим и сплиты заметнее снижают число стадий там, где встречаются редкие, очень объёмные таблицы; при этом возрастают время работы программы размещения и сложность представления решения из-за увеличения числа узлов после разрезания.

Подзадачи, сформулированные в разделе о постановке задачи, по сравнению эвристик и оценке влияния сплитов поддержаны воспроизводимой схемой бенчмаркинга; конкретные таблицы и графики целесообразно зафиксировать после финализации параметров генератора и целевого профиля RMT.

8 Заключение

В работе рассмотрена задача размещения логических match-action таблиц P4 по стадиям конвейера архитектуры RMT при учёте графа зависимостей и аппаратных лимитов памяти.

Сформулированы цель и научно-технические задачи по анализу жадных эвристик FFD, FFL, FFLS и MCF, разработке многопроходного FFD с поддержкой горизонтального разрезания таблиц, реализации FFLS как эталона сравнения, учёту четырёх типов зависимостей в TDG и экспериментальному сравнению по числу стадий и времени работы.

В обзорном разделе показана связь задачи с расписанием на конвейере с ограниченными ресурсами, описаны свойства рассматриваемых жадных методов и выполнена явная декомпозиция исходной проблемы на подзадачи построения TDG, топологической стратификации, проверки допустимости стадии, жадного выбора и обработки переполнения.

Предложено программное решение на языке Python с модульной архитектурой (ввод и валидация TDG, ресурсный учёт, ядра эвристик, сплиттер, отчётность). Оценён порядок асимптотической сложности наивной реализации проверок размещения.

Описана методика синтетических экспериментов и качественно обсуждены ожидаемые различия между FFD и FFLS в зависимости от плотности и типов ограничений в графе.

Практическая значимость работы заключается в том, что построенная схема позволяет систематически сравнивать жадные алгоритмы размещения при проектировании и настройке компиляционного конвейера для программируемых сетевых устройств. Направления дальнейшей работы включают привязку модели ресурсов к конкретной микроархитектуре целевого чипа и сравнение с промышленным компилятором на реальных P4-программах; пост-жадную оптимизацию (локальный перенос между стадиями, симулированный отжиг на окне узлов TDG, ограниченное целочисленное программирование на кластерах связанных таблиц); формализацию аппроксимационных свойств для упрощённых подклассов графов TDG для обоснования порядка обхода в схемах типа FFLS.

Список литературы

- [1] P4: Programming Protocol-Independent Packet Processors / Pat Bosshart, Dan Daly, Glen Gibb et al. // *ACM SIGCOMM Computer Communication Review*. — 2014. — Vol. 44, no. 3. — Pp. 87–95.
- [2] p4c: reference compiler for the P4 language. — GitHub. — дата обращения: 11.05.2026. <https://github.com/p4lang/p4c>.
- [3] P4 Language Specification. — P4.org. — дата обращения: 11.05.2026. <https://p4.org/specs/>.